# Licence 3 Informatique
## INFO 607
## Travaux Pratiques n°2

---

### Exercice 1 : tri récursif par fusion et dérécursification

On considère le type liste récursive suivant :

**Sorte** Liste-récursive ;
**Utilise** Elément, Booléen ;

### Opérations

| | | |
|---|---|---|
| créer-liste | : | $\rightarrow$ Liste-récursive ; |
| vide | : | Liste-récursive $\rightarrow$ Booléen ; |
| premier | : | Liste-récursive $\rightarrow$ Elément ; |
| reste | : | Liste-récursive $\rightarrow$ Liste-récursive ; |
| préfixer | : | Elément $\times$ Liste-récursive $\rightarrow$ Liste-récursive ; |
| suffixer | : | Elément $\times$ Liste-récursive $\rightarrow$ Liste-récursive ; |

**Fin-Sorte** ;

**Q 1.1** Implémentez les fonctions ci-dessus.

On veut réaliser un tri par fusion sur des listes récursives. Ceci implique la définition préalable des fonctions suivantes dans un style récursif :

**Q 1.2** Ecrivez une fonction FUSION(A,B) qui réalise la fusion de deux listes triées A et B, la liste résultante étant elle-même triée.

**Q 1.3** Ecrivez une fonction SSC(A) définissant la première sous-suite croissante dans une liste A. Par exemple, si le type Elément est constitué des lettres de l'alphabet et que l'on considère l'ordre alphabétique, on peut écrire :

SSC(<a,e,f,c,g,h>) = <a,e,f>  et  SCC(<e,b,n,j,y,g>) = <e>

**Q 1.4** Ecrivez une fonction COMP(A,B) définissant la sous-suite complément de la suite A dans la suite B (on suppose que A commence B). Par exemple :

COMP(<r,y,f>, <r,y,f,n,d,z>) = <n,d,z>

**Q 1.5** Ecrivez la fonction de tri TRI(A) en utilisant les fonctions définies précédemment.

**Q 1.6** Dérécursifiez les fonctions FUSION, SSC, COMP et TRI.

**Q 1.7** Comparez expérimentalement les performances de ce tri en version récursive et itérative.

## Exercice 2 : un cas d'application de l'algorithme de Rabin-Karp

On souhaite tester l'efficacité de l'algorithme de Rabin-Karp (vu en cours) dans un cas particulier où les fichiers et les motifs recherchés dans ces derniers sont uniquement constitués de séquences de symboles représentant des digits {"0", …, "9"}.

**Q 2.1** Dans le contexte décrit ci-dessus, implémentez l'algorithme *brute-force* (vu en cours).

Pour implémenter l'algorithme de Rabin-Karp, il est nécessaire d'appliquer une fonction de hachage sur les motifs recherchés ainsi que sur les sous-motifs figurant dans les fichiers. L'idée principale retenue ici est d'associer le motif "XYZTWXVZ" à l'entier naturel XYZTWXVZ. Ainsi, à la chaîne de digits "534872941" correspond l'entier 534872941. On peut alors appliquer une fonction sur l'entier en question (par exemple un modulo) pour obtenir la valeur de hachage associée à la chaîne de digits.

**Q 2.2** Définissez une fonction permettant de calculer la valeur de hachage associée à une chaîne de digits quelconque. A partir de la valeur de hachage d'un sous-motif figurant dans le fichier à une position donnée, proposez un mode de calcul économique permettant de déduire celle du sous-motif correspondant à un décalage d'un cran vers la droite. Implémentez l'algorithme de Rabin-Karp en vous basant sur ces fonctionnalités.

**Q 2.3** Comparez expérimentalement les performances de l'algorithme *brute-force* et de celles de la version de l'algorithme de Rabin-Karp que vous avez implémenté.